```
$ vi notes.txt
$ grep -n 'pattern' file
$ awk '{print $1}' data
~
~
:wq
```

UNIX SYSTEMS
Exam Revision Guide

Semester VI | Unit 5: The VI Editor
Unit 6: Advanced File Searching with GREP and AWK

STRICTLY UNIX • NOT Linux-flavoured

Prepared for exam revision • Analogy-driven, table-based notes
Cross-check against the current MAKAUT syllabus before relying on this for exams.

@Indra

Table of Contents

UNIT 5

The VI Editor

Unix's native, always-available text editor — no mouse, no menus, pure keyboard.

1. What is VI? The Big Picture

VI ("Visual Editor") is the standard text editor that ships with every Unix system. Before GUIs existed, system administrators needed a way to edit configuration files, scripts, and source code directly from a terminal — vi was built to do exactly that, fast and without depending on a mouse.

ANALOGY

Think of vi as a **manual transmission car**. A GUI editor like Notepad is an automatic — point, click, type. Vi is manual — you choose every gear (mode) yourself. It feels harder at first, but once mastered, it is far quicker because your fingers never leave the keyboard.

VI is a **modal editor**. This is the single most important idea to understand before anything else: at any given moment, vi is in exactly one of three modes, and the same key does completely different things depending on which mode you're in. Pressing `i` in Command mode starts inserting text; pressing `i` while already in Insert mode simply types the letter "i" into the document.

UNIX NOTE

On a genuine Unix system (Solaris, HP-UX, AIX, BSD) the editor invoked by typing `vi` is the original vi. Many Linux distributions actually alias `vi` to **vim** (Vi IMproved), which adds extra features. For exam purposes, stick to the classic vi commands listed in this guide — they work identically on true Unix and are a safe subset of vim too.

2. The Three Modes of VI

Every action in vi happens inside one of three modes. Switching between them correctly is 90% of learning vi.

Mode	Purpose	How to Enter	How to Leave
Command Mode	The default mode. Used for moving the cursor, deleting, copying, and launching other modes. Keystrokes are interpreted as commands, not as text.	Default mode on opening vi; press Esc from any other mode	Press i/a/o etc. to go to Insert, or : to go to Last-line
Insert Mode	Used to actually type and edit text content, exactly like a normal text editor.	From Command mode: i, I, a, A, o, or O	Press Esc to return to Command mode
Last-line Mode (Ex mode)	Used for file-level operations: saving, quitting, search-replace, line jumps. Commands appear on the bottom line of the screen, starting with a colon.	From Command mode, press : (or / or ? for search)	Press Enter to execute, or Esc to cancel

MEMORY TRICK

Remember the flow as a triangle: **Command is HOME.** From Command mode you can travel to Insert mode (to type) or Last-line mode (to give file commands), but Insert and Last-line never talk to each other directly — you must press **Esc** and return Home (Command mode) first.

Quick Visual: The Mode Triangle

```
$ mode-flow.txt
i, I, a, A, o, O
COMMAND MODE --------------------> INSERT MODE
(Esc returns) <-------------------- (Esc to leave)
| ^
| |
: | | Esc
v |
LAST-LINE MODE (:w :q :wq /pattern :s/old/new/)
```

3. Starting VI and Opening Files

You start vi from the Unix shell prompt by typing **`vi`** followed by a filename.

```
$ terminal
$ vi filename
# Opens 'filename' for editing.
# If the file does not exist, vi creates a NEW empty buffer
# with that name -- the file is only actually written to disk
# once you save it with :w
$ vi
# Opens vi with no filename -- an unnamed empty buffer.
# You will be asked to supply a name later, using :w filename
```

Example walk-through: editing a Unix shell script called `backup.sh`

```
$ terminal
$ vi backup.sh
# vi opens. The screen shows the file's contents (or, if new,
# an empty screen with ~ symbols marking lines with no content).
# The cursor starts at the top-left, and vi is in COMMAND MODE.
```

UNIX NOTE

Those tilde (~) characters down the left margin are not part of your file. They simply mean "this is past the end of the file" — a vi convention so you can see where your actual content ends.

4. Navigation — Moving the Cursor

Navigation keys only work in **Command mode**. The classic vi keyboard does not always have arrow keys configured correctly over a remote terminal connection — which is exactly why the h, j, k, l keys exist: they sit right under your fingers on a standard keyboard row.

MEMORY TRICK

Picture the four letters **h j k l** sitting in a row on the keyboard. h is on the far left, l is on the far right — so naturally, **h = left** and **l = right**. The letter j has a descender, like an arrow drooping down — so **j = down**. That leaves **k = up**, the one above j.

Key	Moves Cursor
`h (or Left arrow)`	One character to the LEFT
`j (or Down arrow)`	One line DOWN
`k (or Up arrow)`	One line UP
`l (or Right arrow)`	One character to the RIGHT

Moving by Word, Line, and Page

Command	Effect
`w`	Jump forward to the start of the NEXT word
`b`	Jump backward to the start of the PREVIOUS word
`e`	Jump to the END of the current/next word
`0 (zero)`	Jump to the very FIRST character of the current line
`$`	Jump to the LAST character of the current line
`G`	Jump to the LAST line of the file
`gg (or 1G)`	Jump to the FIRST line of the file
`nG`	Jump to line number n (e.g. 25G jumps to line 25)
`Ctrl+f`	Scroll forward (DOWN) one full screen/page
`Ctrl+b`	Scroll backward (UP) one full screen/page

UNIX NOTE

On many genuine Unix terminals (especially over a slow serial line or an older terminal type setting), arrow keys can send escape sequences that vi does not recognise correctly. h/j/k/l are the guaranteed-portable way to move in true Unix vi — always learn them first.

5. Entering Insert Mode

There are six different ways to enter Insert mode from Command mode — each places your cursor at a different starting point, which saves you extra navigation keystrokes.

Key	Enters Insert Mode... Where?
`i`	Insert BEFORE the character under the cursor
`I`	Insert at the BEGINNING of the current line
`a`	Append AFTER the character under the cursor
`A`	Append at the END of the current line
`o`	Open a NEW line BELOW the current line, and insert there
`O`	Open a NEW line ABOVE the current line, and insert there

MEMORY TRICK

Lowercase vs uppercase tells you "how far": lowercase i/a stay close to the cursor (before/after one character); uppercase I/A jump to an extreme of the line (start/end). Same logic for o/O: lowercase o opens a line below (the natural direction text flows), uppercase O opens one above.

Worked Example

Suppose a file contains this single line, and the cursor (shown as `[ ]`) sits on the letter **`U`**:

```
[U]nix is powerful
```

• Pressing **`i`** then typing **`Real `** gives: **`Real Unix is powerful`** (text inserted BEFORE the U)

• Pressing **`a`** then typing **`nique`** gives: **`Unique nix is powerful`** (text inserted AFTER the U)

• Pressing **`A`** then typing **`, truly.`** gives: **`Unix is powerful, truly.`** (appended at line end, cursor position doesn't matter)

• Pressing **`o`** then typing **`It rocks.`** creates a brand new line below, containing: **`It rocks.`**

After any of these, press **`Esc`** to return to Command mode and lock the changes into the buffer.

6. Deleting Text

All deletion commands below are typed in **Command mode** — do not press i first.

Command	Deletes
`x`	ONE character under the cursor (forward delete)
`X`	ONE character BEFORE the cursor (backward delete, like Backspace)
`dd`	The ENTIRE current line
`ndd (e.g. 3dd)`	n entire lines, starting from the current line
`dw`	From the cursor to the end of the current word
`D`	From the cursor to the END of the current line (same as d$)
`d0`	From the cursor to the START of the current line
`dG`	From the current line to the END of the file

MEMORY TRICK

Most deletion commands in vi follow the pattern **d + motion** — 'd' simply means "delete," and whatever movement key follows tells it HOW FAR to delete. **dw** = delete-word, **d$** = delete-to-end-of-line, **dG** = delete-to-end-of-file. Once you know the motion keys from Section 4, you already know most delete commands too!

UNIX NOTE

In true Unix vi, every deleted chunk of text is automatically placed into an internal buffer (sometimes called the "unnamed register"), which is exactly what makes delete-then-paste work — see Section 7.

Example: cleaning up a config line

```
Before: PORT=8080 # temporary value, remove this comment
# Cursor placed on the '#'. Pressing D deletes from cursor to line end:
After: PORT=8080
```

7. Copying and Pasting (Yank and Put)

Unix vi does not use the word "copy" — it calls this operation **yank**. Pasting is called **put**.

Command	Action
`yy`	Yank (copy) the CURRENT line into the buffer
`nyy (e.g. 3yy)`	Yank n lines starting from the current line
`yw`	Yank from the cursor to the end of the current word

Command	Action
p	Put (paste) the yanked/deleted text AFTER the cursor / below the current line
P	Put (paste) the yanked/deleted text BEFORE the cursor / above the current line

MEMORY TRICK

Lowercase **p** pastes AFTER/BELOW, uppercase **P** pastes BEFORE/ABOVE — exactly the same lowercase/uppercase logic you already used for i/a and o/O. Once you see this pattern in vi, you stop memorising individual commands and start memorising one simple rule.

Note from your topic list: 'pp' simply means pressing **p** twice — pasting the same yanked line twice in a row, once immediately after the other.

Worked Example

```
Line 1: export PATH=/usr/bin
Line 2: export HOME=/home/indra

# Cursor on Line 1. Press yy -> Line 1 is copied (not removed).
# Move cursor to Line 2. Press p -> pastes copy AFTER Line 2:

Result:
Line 1: export PATH=/usr/bin
Line 2: export HOME=/home/indra
Line 3: export PATH=/usr/bin <- pasted copy
```

8. Undo and Redo

Command	Action
`u`	Undo the LAST change made
`Ctrl+r`	Redo — reverse the last undo ("redo" what you just undid)

UNIX NOTE

Classic original Unix vi historically only guaranteed ONE level of undo — pressing **u** repeatedly could simply toggle between the undone and redone state rather than walking back through many changes. Multiple-level undo and Ctrl+r as "redo" are vim enhancements that most modern Unix systems now ship with by default, but for strict exam answers, mention that the ORIGINAL vi standard guarantees only a single undo step.

9. Saving and Exiting Files

These are all **Last-line mode** commands — type a colon first, then the command, then press Enter.

Command	Effect
`:w`	Write (save) the file, keeping vi open, under its CURRENT filename
`:w filename`	Write (save) the buffer's contents into a NEW file called filename, without changing what file vi is currently editing
`:q`	Quit vi -- only works if there are NO unsaved changes
`:q!`	Force-quit WITHOUT saving, discarding all unsaved changes
`:wq`	Write (save), then quit -- the most common way to finish editing
`ZZ`	Shortcut for save-and-quit, typed in COMMAND mode (no colon, no Enter)

MEMORY TRICK

Think of the exclamation mark ! across all of Unix as meaning "I really mean it, override any warning." `:q!` = "quit, and yes, I know I'll lose my changes, do it anyway." You'll see this same ! pattern reused elsewhere in Unix commands.

ZZ vs :wq — both save and quit, but ZZ is typed directly in Command mode (two capital Z's, no Enter needed afterwards), while :wq is typed in Last-line mode and ends with Enter. ZZ is simply a faster shortcut for the same outcome.

10. File Operations — Opening a New File

Command	Effect
`:e filename`	Edit (open) a DIFFERENT file inside the same vi session, replacing the current buffer view
`:e!`	Re-edit (reload) the current file from disk, discarding unsaved in-buffer changes -- a quick "reset"
`:r filename`	Read another file's contents and INSERT it into the current buffer at the cursor position (does not replace, just adds)

UNIX NOTE

:e filename will refuse to switch files if you have unsaved changes, exactly the same safety behaviour as plain :q. Save first with :w, or force it using :e! filename.

Worked Example

```
$ inside vi (last-line mode commands)
$ vi notes.txt
# ... editing notes.txt ...
:w
# saves notes.txt
:e todo.txt
# now editing todo.txt in the SAME vi session, no need to quit vi first
```

11. Search and Replace

11.1 Searching Forward and Backward

Command	Effect
`/pattern`	Search FORWARD (downward) through the file for "pattern"
`?pattern`	Search BACKWARD (upward) through the file for "pattern"
`n`	Repeat the last search in the SAME direction (next match)
`N`	Repeat the last search in the OPPOSITE direction

MEMORY TRICK

The forward-slash / always points down on a number-line mentally and ? simply means "the other direction" — / searches down/forward through the file, ? searches up/backward.

11.2 Using Regular Expressions in Search

Vi's search accepts basic regular expressions, not just plain text — letting you match patterns rather than exact words.

Pattern	Matches
`^word`	Lines that START WITH "word" (^ anchors to line beginning)
`word$`	Lines that END WITH "word" ($ anchors to line end)
`.`	Any single character (wildcard)
`*`	Zero or more repetitions of the PREVIOUS character
`[abc]`	Any ONE character that is a, b, or c
`[^abc]`	Any ONE character that is NOT a, b, or c

```
$ inside vi
/^Error
# Finds the next line that begins with the word 'Error'
/[0-9]*$
# Finds lines ending in zero or more digits
```

11.3 Replace — :s/old/new/

The substitute command is one of the most powerful tools in vi. Its general form is:

```
$ general syntax
:[range]s/old-pattern/new-pattern/[flags]
```

Example Command	What It Does
`:s/cat/dog/`	Replace the FIRST occurrence of "cat" with "dog" on the CURRENT line only
`:s/cat/dog/g`	Replace ALL occurrences of "cat" with "dog" on the CURRENT line (the g flag = global, within that line)
`:%s/cat/dog/g`	Replace ALL occurrences of "cat" with "dog" across the ENTIRE FILE -- the most commonly used form
`:%s/cat/dog/gc`	Same as above, but ASK FOR CONFIRMATION before each replacement (c = confirm)
`:5,10s/cat/dog/g`	Replace ALL occurrences only within LINES 5 through 10

MEMORY TRICK

Read **`:%s/old/new/g`** out loud as a sentence: "**%** = every line in the file, **s** = substitute, **old/new** = what to find and replace it with, **g** = do it globally (every match per line, not just the first)." This single command is the one examiners ask about most often -- memorise it as one phrase, not five separate symbols.

11.4 Global Search and Replace — Full Worked Example

Suppose a file **`report.txt`** contains:

```
The Unix system is reliable.
I love the Unix shell.
Unix was created at Bell Labs.
```

To replace every occurrence of "Unix" with "UNIX" throughout the whole file:

```
$ inside vi, last-line mode
:%s/Unix/UNIX/g
```

Result:

```
The UNIX system is reliable.
I love the UNIX shell.
UNIX was created at Bell Labs.
```

Finally, save and quit using **`:wq`** or **`ZZ`**.

UNIT 6

GREP and AWK

Advanced file searching and pattern-driven text processing on Unix.

12. grep — Pattern Searching

grep stands for **G**lobal **R**egular **E**xpression **P**rint. It scans through one or more files, line by line, and prints every line that matches a given pattern.

ANALOGY

Think of grep as a librarian with a highlighter, given a giant book (your file) and one instruction: "highlight and read out every single sentence (line) that contains the word I tell you." It never edits the book — it just reports matching lines back to you.

Basic Syntax

$ general syntax

```
grep 'pattern' filename
```

$ terminal

```
$ cat marks.txt
Indra 78
Rahul 65
Sneha 91
$ grep 'Rahul' marks.txt
Rahul 65
# Only the matching line is printed -- the rest of the file is untouched.
```

13. Advanced grep — Useful Options and Regex

Command / Option	What It Does
`grep -i 'pattern' file`	Case-INSENSITIVE search (matches Pattern, PATTERN, pattern, all the same)
`grep -v 'pattern' file`	INVERT match -- print every line that does NOT contain the pattern
`grep -n 'pattern' file`	Show the LINE NUMBER alongside each matching line
`grep -c 'pattern' file`	Show only the COUNT of matching lines, not the lines themselves

Command / Option	What It Does
`grep -l 'pattern' *.txt`	Across MULTIPLE files, list only the FILENAMES that contain a match
`grep -r 'pattern' dir/`	RECURSIVELY search every file inside a directory tree
`grep '^A' file`	Lines STARTING with A (regex anchor)
`grep 'A$' file`	Lines ENDING with A (regex anchor)
`grep '[0-9]' file`	Lines containing AT LEAST ONE digit
`grep -E 'cat|dog' file`	EXTENDED regex -- match lines containing "cat" OR "dog"

UNIX NOTE

The -E flag (extended regular expressions, enabling things like the | alternation operator directly) is part of POSIX and present on genuine Unix grep. Without -E, plain grep interprets | as a literal character, not an OR operator -- a common exam trick question.

14. awk — The Big Picture

awk is named after its three original authors -- **A**ho, **W**einberger, and **K**ernighan. Unlike grep, which only finds lines, awk is a full pattern-scanning and data-processing language: it reads a file line by line, automatically splits each line into fields, and lets you run actions on those fields.

> **ANALOGY**
> If grep is a librarian who only reads out matching sentences, awk is an accountant who takes every row of a ledger, automatically splits it into columns (name, amount, date...), and lets you calculate, compare, and reformat those columns on the fly.

General Syntax

```
$ general syntax
awk 'pattern { action }' filename
# pattern -- a condition deciding WHICH lines to act on (optional)
# action -- what to DO to those lines, in { curly braces }
```

awk automatically splits each input line into **fields**, referred to as **$1**, **$2**, **$3**, and so on. **$0** always means the WHOLE line.

15. awk print and printf

Consider this sample data file used throughout the rest of this unit:

```
$ terminal
$ cat marks.txt
Indra 78 85
Rahul 65 70
Sneha 91 88
# Field layout: $1 = Name $2 = Marks1 $3 = Marks2
```

15.1 print

```
$ terminal
$ awk '{print $1}' marks.txt
Indra
Rahul
Sneha
# Prints ONLY the first field (the name) from every line
$ awk '{print $1, $2}' marks.txt
Indra 78
Rahul 65
```

```
Sneha 91
# Comma in print inserts a single space between fields
```

15.2 printf — Formatted Output

printf works like print, but gives precise control over spacing, decimal places, and alignment using format specifiers such as **%s** (string), **%d** (integer), and **%f** (floating point).

$ terminal

```
$ awk '{printf "%-10s %d\n", $1, $2}' marks.txt
Indra 78
Rahul 65
Sneha 91
# %-10s = left-align the string in a 10-character-wide field
# %d = print as an integer
# \n = newline -- printf does NOT add one automatically, unlike print
```

UNIX NOTE

This is the single biggest gotcha between print and printf: print automatically starts a new line after each statement. printf does NOT -- you must add \n yourself, or all your output runs together on one line.

16. awk Comparison Operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

$ terminal

```
$ awk '$2 > 70 {print $1}' marks.txt
Indra
Sneha
# Pattern $2 > 70 acts as a FILTER: only lines where field 2 exceeds
# 70 trigger the action -- here, printing the name.
```

17. awk Arithmetic Operators

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus (remainder)
`^ (or **)`	Exponent (power)

$ terminal

```
$ awk '{print $1, $2+$3}' marks.txt
Indra 163
Rahul 135
Sneha 179
# Adds field 2 and field 3 together and prints alongside the name
$ awk '{print $1, ($2+$3)/2}' marks.txt
Indra 81.5
Rahul 67.5
Sneha 89.5
# Computes the average of the two marks
```

18. BEGIN and END Sections

Normally, the action block in awk runs once FOR EVERY LINE of input. But sometimes you need code that runs only ONCE -- before any line is read, or only ONCE after all lines are finished. That is exactly what BEGIN and END are for.

Block	Runs
`BEGIN { ... }`	ONCE, BEFORE any input line is read -- typically used to print headers, or initialise variables
`{ ... }`	Once FOR EVERY input line -- the main, repeating action
`END { ... }`	ONCE, AFTER all input lines have been processed -- typically used to print totals, summaries, or footers

ANALOGY

Think of an awk script as a three-course meal: BEGIN is the starter (served once, before the main course begins), the middle block {...} is the main course (served once per guest, i.e. once per line), and END is the dessert (served once, after every guest has been fed).

Worked Example — Header, Body, and Total

$ terminal

```
$ awk 'BEGIN {print "NAME\tTOTAL"}
{print $1"\t"($2+$3); sum += $2+$3}
END {print "-----------------"; print "Grand Total:", sum}' marks.txt
# Output:
NAME TOTAL
Indra 163
Rahul 135
Sneha 179
-----------------
Grand Total: 477
```

Note how the variable **sum** is declared nowhere -- awk creates variables automatically the first time they're used, starting at zero, which is why **sum += $2+$3** works correctly from the very first line.

19. if-else Statement in awk

awk supports standard if-else conditional logic inside the action block, just like Java or C, letting you branch behaviour based on field values.

```
$ general syntax
if (condition) {
statement(s) if true
} else {
statement(s) if false
}
```

Worked Example — Pass/Fail Grading

```
$ terminal
$ awk '{
avg = ($2+$3)/2
if (avg >= 75) {
print $1, "PASS with distinction"
} else if (avg >= 40) {
print $1, "PASS"
} else {
print $1, "FAIL"
}
}' marks.txt
# Output:
Indra PASS with distinction
Rahul PASS
Sneha PASS with distinction
```

20. Built-in Variables: FS and OFS

Variable	Stands For	Default	Purpose
FS	Field Separator	Space / Tab	Tells awk how INPUT fields are separated (split) when reading each line
OFS	Output Field Separator	Single space	Tells awk what character to place BETWEEN fields when it builds output using print with commas

MEMORY TRICK

FS controls how awk reads a line APART, OFS controls how awk puts fields back TOGETHER when printing. "F" for the input split, "OF" (Output Field) for the output join -- the names directly describe their job once you notice the IN vs OUT direction.

Worked Example — Reading a Colon-Separated File

Many genuine Unix system files (like /etc/passwd) use a colon ':' instead of a space to separate fields. To process such a file correctly, FS must be changed:

```
$ terminal
$ cat users.txt
indra:20:student
rahul:21:student
$ awk -F: '{print $1, $2}' users.txt
indra 20
rahul 21
# -F: on the command line is a shortcut for setting FS=":"
$ awk 'BEGIN {OFS="-"} {print $1, $2}' users.txt
indra-20
rahul-21
# OFS changes the separator used when JOINING fields with a comma in print
```

21. String and Arithmetic Functions

21.1 Common String Functions

Function	Purpose
`length(str)`	Returns the NUMBER OF CHARACTERS in str (or of $0 if no argument given)
`substr(str, start, len)`	Extracts a SUBSTRING of str, starting at position start, for len characters
`toupper(str)`	Converts str to ALL UPPERCASE
`tolower(str)`	Converts str to all lowercase
`index(str, target)`	Returns the POSITION at which target first appears inside str (0 if not found)
`split(str, array, sep)`	Splits str into pieces using separator sep, storing pieces into array

```
$ terminal
$ awk '{print toupper($1), length($1)}' marks.txt
INDRA 5
RAHUL 5
SNEHA 5
```

21.2 Common Arithmetic (Math) Functions

Function	Purpose
`sqrt(x)`	Square root of x
`int(x)`	Integer part of x (truncates decimals, does not round)
`exp(x)`	e raised to the power x
`log(x)`	Natural logarithm of x
`rand()`	Random number between 0 and 1

22. Loops in awk

awk supports the same loop types found in C/Java -- for, while, and do-while -- useful for repeating an action a fixed number of times, e.g. looping over fields you don't know the count of in advance.

Loop Type	Syntax
`for`	for (i = 1; i <= NF; i++) { ... }
`while`	while (condition) { ... }

Loop Type	Syntax
do-while	do { ... } while (condition)

UNIX NOTE

NF is another built-in awk variable -- it means "Number of Fields" in the CURRENT line. Looping with for (i=1; i<=NF; i++) is the standard way to walk through EVERY field of a line, regardless of how many fields that particular line happens to have.

Worked Example — Print Every Field, One Per Line

$ terminal

```
$ awk '{ for (i = 1; i <= NF; i++) print i, $i }' marks.txt
# For the FIRST line (Indra 78 85), output is:
1 Indra
2 78
3 85
# ... and the loop repeats for every subsequent line, too.
```

23. Search and Substitute Functions

awk has its own built-in functions for pattern matching and text replacement, distinct from (but similar in spirit to) the regex work you already did inside vi in Unit 5.

Function	Purpose
`match(str, regex)`	Tests whether regex is found anywhere inside str. Returns the STARTING POSITION of the match (0 if no match)
`sub(regex, replacement, target)`	Replaces ONLY the FIRST match of regex within target with replacement
`gsub(regex, replacement, target)`	Replaces ALL matches of regex within target with replacement (g = global)

MEMORY TRICK

sub vs gsub follows the exact same logic as vi's :s vs :s/.../g from Unit 5 -- **sub = first match only**, **gsub = every match, globally**. If you remember the vi rule, you already know the awk rule too.

Worked Example

$ terminal

```
$ awk '{gsub(/Indra/, "I.Ganguly"); print}' marks.txt
# Every occurrence of 'Indra' in each line is replaced by 'I.Ganguly'
I.Ganguly 78 85
Rahul 65 70
Sneha 91 88
$ awk '{print match($0, /[0-9]+/)}' marks.txt
# Prints the position where the first run of digits starts on each line
10
10
10
```

Final Quick-Revision Sheet

A one-glance cheat-sheet covering both units -- ideal for last-minute revision before walking into the exam hall.

VI Editor — At a Glance

Goal	Command
`Open a file`	vi filename

Goal	Command
Move cursor	h j k l / w b / 0 $ / gg G
Insert text	i I a A o O
Delete	x dd dw D
Copy / Paste	yy / p P
Undo / Redo	u / Ctrl+r
Save	:w
Save & quit	:wq or ZZ
Quit without saving	:q!
Open another file	:e filename
Search forward / backward	/pattern ?pattern
Replace globally	:%s/old/new/g

grep & awk — At a Glance

Goal	Command
Find lines with a pattern	grep 'pattern' file
Case-insensitive search	grep -i 'pattern' file
Invert match / line numbers	grep -v ... / grep -n ...
Print a field	awk '{print $1}' file
Formatted output	awk '{printf "%s %d\n", $1, $2}' file
Filter by condition	awk '$2 > 70 {print $1}' file
Run once before/after	BEGIN {...} / END {...}
Change input separator	awk -F: '{...}' file
Loop over fields	for (i=1; i<=NF; i++) ...
Replace first / all matches	sub(...) / gsub(...)

REMINDER

As you noted yourself -- always cross-check this against the current official MAKAUT syllabus PDF before your exam, since university syllabi are occasionally revised. This guide covers Semester 6 topics exactly as you listed them.